1. Project Summary

Apex solves the "I know what infrastructure I want, but writing safe CDK from scratch is slow" problem. A user enters a plain-English AWS infrastructure request; Apex generates TypeScript CDK with Anthropic, synthesizes it in a restricted sandbox Lambda, analyzes the resulting CloudFormation template for diff/cost/security, stores the generation, waits for user approval, then deploys supported resources through CloudFormation.

What you built: a Next.js frontend plus a serverless AWS backend with Lambda orchestration, DynamoDB generation history, S3 template storage, WebSocket progress streaming, CloudFormation change-set deployment, and a reusable @apex/changeset analysis package.

45-second answer:

> *Apex is an AI-powered DevOps copilot for AWS infrastructure. The user describes infrastructure in natural language, and the system generates TypeScript CDK using Anthropic, validates it by running cdk synth inside a restricted Lambda sandbox, then shows a resource diff, cost estimate, and security warnings before anything is deployed. After approval, a deploy Lambda uploads the synthesized CloudFormation template to S3, creates and executes a CloudFormation change set under a scoped deployment role, streams live deployment events over WebSockets, and stores the final state in DynamoDB. I built it as a serverless architecture to keep operations simple and to make the safety boundary clear between AI generation, validation, review, and real AWS deployment.*

2. Architecture

```
Next.js UI
-> Orchestration Lambda Function URL
-> Secrets Manager -> Anthropic API
-> Sandbox Lambda -> cdk synth -> CloudFormation template
-> changeset package -> diff + cost + security
-> DynamoDB: generation status awaiting_approval
-> WebSocket API: pipeline_step events

User clicks Approve
-> API Gateway REST /approve
-> Approval Lambda -> DynamoDB status deploying
-> async Invoke Deploy Lambda
-> S3 template upload
-> CloudFormation Change Set with DeploymentRole
-> poll stack events
-> WebSocket deploy_event logs
-> DynamoDB deployed/deploy_failed
```

Request walkthrough: in page.tsx, handleGenerate posts { message, conversationId, connectionId } to the orchestration URL. orchestrate/index.ts writes generating, calls Anthropic, invokes sandbox, analyzes, stores awaiting_approval, and returns code/diff/cost/security. On approval, approve/index.ts writes deploying and async-invokes deploy. deploy/index.ts strips CDK bootstrap metadata, blocks unsupported resource types, uploads the template to S3, creates/executes a change set, streams events, and writes deployed or deploy_failed.

3. Tech Stack

Tech	What / used here	Why chosen	Alternative
Next.js + React + TS	Single-page UI, prompt form, history, panels in page.tsx	Fast typed frontend	Vite/React, Remix
Tailwind CSS	Utility styling in components	Quick consistent UI	CSS modules, MUI
xterm.js	Live deploy terminal in DeployLogPanel	Terminal-like CFN logs	Plain log list
AWS CDK	Defines all AWS infra in infra-stack.ts	Infra as code in TS	Terraform, SAM
Lambda	Orchestrate, sandbox, approve, deploy, history, WS handlers	Serverless, isolated steps	ECS/Fargate, Step Functions

Tech	What / used here	Why chosen	Alternative
Function URL	Main long-running orchestration endpoint, 120s	Avoid API Gateway 29s timeout	Async job API, Step Functions
API Gateway REST	/approve, /history, legacy routes	Simple HTTP APIs	Lambda Function URLs only
API Gateway WebSocket	Pipeline/deploy events	Real-time progress	SSE, polling
DynamoDB	Composite key conversationId + generationId	Serverless state/history	Postgres, RDS
S3	Stores deploy templates	CFN can read template URLs	Inline templates
CloudFormation	Real deployment through change sets	Native AWS deployment engine	CDK deploy inside Lambda
IAM	DeploymentRole allowlist and PassRole scoping	Safety boundary	Admin role, SCPs
Secrets Manager	Anthropic key storage	Avoid plaintext env secret	SSM Parameter Store
Anthropic SDK	Claude CDK generation	Strong code generation	Bedrock, OpenAI
Zod	Runtime request/response validation	Defensive parsing for LLM/API data	io-ts, manual checks
Jest	Unit/CDK assertion tests	Fast backend validation	Vitest

4. Important Code

- infra-stack.ts: wires DynamoDB, S3, Lambdas, API Gateway, WebSocket API, IAM, outputs.
- orchestrate/index.ts: core pipeline: request validation, generation, sandbox synth, analysis, DynamoDB state.
- sandbox/index.ts: isolated cdk synth runner; temp workspace, fake AWS env, 50s timeout.
- deploy/index.ts: real deploy engine; S3 template upload, change sets, polling, rollback/failure handling.
- packages/changeset/src/analyzer.ts: combines template diff, heuristic cost estimate, and security scanner.

5. Technical Decisions

- I used a Lambda Function URL for generation because Anthropic plus sandbox synth can exceed API Gateway's 29-second limit; the tradeoff is that auth/CORS hardening is still needed.
- I split approval and deploy into separate Lambdas so the HTTP approval request returns quickly while deployment can run up to 15 minutes asynchronously.
- I ran CDK synth in a separate Lambda with log-only permissions, so generated code can be validated without AWS API permissions. It's isolation for synth validation, not a full container security product.
- I used DynamoDB with conversation/generation keys because the data is naturally lookup-by-conversation and schemaless enough for evolving deployment metadata.
- I deploy through CloudFormation change sets under a scoped DeploymentRole rather than giving the AI-generated template broad credentials; the tradeoff is only S3/KMS/CDK metadata resources are deployable today.

6. Interview Questions

1. Q: What does Apex do?

A: It turns natural language into CDK, validates via sandbox synth, reviews diff/cost/security, then deploys approved templates.

Follow-up: Is it production-ready?

A: The generated CDK aims for production-style defaults, but the app still needs auth, rate limiting, and CORS hardening.

2. Q: Walk me through the architecture.

A: UI -> Orchestration Lambda -> Anthropic/Sandbox/Analyzer -> DynamoDB/WebSocket -> Approval Lambda -> Deploy Lambda -> S3/CloudFormation.

Follow-up: Why WebSockets?

A: Deployments are long-running, so WebSockets give live progress without polling.

3. Q: Why Lambda?

A: It fits bursty, event-driven generation/deploy tasks and keeps ops low.

Follow-up: Limitation?

A: 15-minute max runtime; Step Functions or ECS would handle longer workflows better.

4. Q: Why Function URL instead of API Gateway for generation?

A: API Gateway REST times out around 29s; orchestration can take up to 120s.

Follow-up: Risk?

A: It is unauthenticated with open CORS today, so I'd add auth/throttling before production.

5. Q: How is generated code validated?

A: Sandbox Lambda writes code to /tmp, symlinks CDK deps from a layer, runs cdk synth, and returns the template.

Follow-up: Is it fully sandboxed?

A: It has a restricted IAM role and temp workspace, but not a hardened container jail.

6. Q: How do you prevent dangerous deployments?

A: Manual approval, security scanner, deployability allowlist, scoped CloudFormation DeploymentRole, and iam:PassRole only to that role.

Follow-up: Gap?

A: S3/KMS role resources are still *; production should scope bucket prefixes where possible.

7. Q: What is stored in DynamoDB?

A: Prompt, generated code/explanation, CloudFormation template, diff, cost, security flags, status, deploy outputs/errors.

Follow-up: Why composite key?

A: conversationId groups history; generationId identifies each generated stack.

8. Q: How does deployment work?

A: Deploy Lambda uploads template to S3, creates a CFN change set using RoleARN, executes it, polls stack events, streams logs, persists terminal state.

Follow-up: Why change sets?

A: They are the native safe CloudFormation mechanism for create/update/no-op detection.

9. Q: What happens on failure?

A: Orchestration writes failed; deploy writes deploy_failed, stores error, streams failure, and avoids stuck deploying.

Follow-up: Retry?

A: Approval accepts deploy_failed and can async-invoke deploy again.

10. Q: How is cost estimated?

A: The analyzer maps supported resource types to heuristic monthly costs and lightly consults Pricing API metadata.

Follow-up: Limitation?

A: It is not a full usage-based estimator.

11. Q: What security checks exist?

A: IAM wildcard action/resource, S3 public access/encryption, RDS encryption, open EC2 security groups.

Follow-up: What's missing?

A: Broader policy analysis, network topology, secrets detection, compliance rules.

12. Q: Why Anthropic direct API instead of Bedrock?

A: Simpler direct integration with SDK and Secrets Manager; no Bedrock account/model setup.

Follow-up: Tradeoff?

A: External provider dependency and separate rate/cost controls.

13. Q: How would you scale it?

A: Move orchestration to async job/Step Functions, add queues, cache generations, persist WS connections, add rate limits, shard by user/account.

Follow-up: DynamoDB scaling?

A: Current on-demand mode scales reads/writes, but add GSIs for user/time queries.

14. Q: Why not run cdk deploy directly?

A: Raw CloudFormation change sets give explicit service role control and easier event polling.

Follow-up: Downside?

A: Need to strip CDK bootstrap metadata and manage template upload manually.

15. Q: What would you improve first?

A: Add auth/Cognito or token auth, lock CORS, throttle APIs, scope deployment role resources, and make cost/security checks deeper.

Follow-up: Why first?

A: The current biggest risk is public unauthenticated endpoints causing cost or unsafe access.

7. Resume Verification

- Claim 1: "Built an AI-powered DevOps copilot that converts natural language prompts into production-ready AWS CDK infrastructure using the Anthropic API." Supported by prompt.ts, orchestrate/index.ts, and page.tsx. Defend it as: "I used Anthropic to generate CDK TypeScript and validated it before review." Flag: "production-ready" is aspirational; better say "production-style CDK with safety defaults."
- Claim 2: "Designed a serverless orchestration pipeline with AWS Lambda, API Gateway, DynamoDB, and S3 to generate, validate, store, and manage infrastructure deployments." Supported by infra-stack.ts, generation.ts, approve/index.ts, deploy/index.ts. Accurate. Nuance: generation mainly uses a Lambda Function URL; API Gateway handles approve/history and legacy routes.
- Claim 3: "Implemented sandboxed CDK validation, infrastructure diff previews, IAM security checks, and AWS cost estimation before deployment to ensure safe infrastructure provisioning." Supported by sandbox/index.ts, parser.ts, securityScanner.ts, costEstimator.ts. Mostly accurate. Flags: diff preview is an internal template diff, not a pre-approval CloudFormation change set; IAM checks are limited to wildcard detection; cost estimation is heuristic; deployability blocking happens after approval but before CloudFormation.
- Suggested tighter resume bullet: "Implemented Lambda-based CDK synth validation, template diff previews, targeted security checks, and heuristic AWS cost estimates before approval; deployed approved S3/KMS templates through scoped CloudFormation change sets."

8. Final Cheat Sheet

- Problem: Writing safe AWS CDK manually is slow; AI-generated infra needs validation and guardrails.
- Architecture: Next.js -> Orchestration Lambda -> Anthropic + Sandbox Lambda + changeset analyzer -> DynamoDB + WebSocket -> Approval Lambda -> Deploy Lambda -> S3 template -> CloudFormation change set with DeploymentRole -> WebSocket logs -> DynamoDB final state.
- Tech stack: Next.js/React/TypeScript, AWS CDK, Lambda, Function URL, API Gateway REST/WebSocket, DynamoDB, S3, CloudFormation, IAM, Secrets Manager, Anthropic SDK, Zod, Jest.
- My contribution: Built the AI-to-CDK orchestration, sandbox validation, analysis package, approval/deploy pipeline, WebSocket logs, and frontend review UI.
- Hardest technical problem: Safely moving from AI-generated code to real AWS deployment; solved with synth sandboxing, manual approval, allowlisted DeploymentRole, deployability guardrail, and CloudFormation event polling.
- 3 decisions: Function URL for long generation; separate async Deploy Lambda; scoped CloudFormation DeploymentRole.
- Biggest tradeoff: Safer narrow deployment support versus broad AWS service coverage.
- How I'd scale it: Step Functions/SQS for orchestration, persisted job status and WS connections, per-user auth, throttling, deeper cost/security engines, broader service allowlist.
- What I'd improve: Auth/Cognito, CORS lockdown, rate limiting, resource-scoped IAM, real CloudFormation preview before approval, production frontend hosting.
- Top 10 quick answers: 1. AI DevOps copilot for CDK generation and guarded deploy. 2. Prompt -> generate -> synth -> analyze -> approve -> deploy. 3. Low ops, event-driven, natural AWS integration. 4. Avoid API Gateway timeout for long generation. 5. Run cdk synth in restricted Lambda sandbox. 6. DynamoDB by conversationId and generationId. 7. S3 template + CloudFormation change set + DeploymentRole. 8. Approval, scanner, allowlist, scoped PassRole. 9. Auth, rate limiting, locked CORS, broad resource deployment are incomplete. 10. Add auth/throttling first, then expand deploy support.

STAR Story

Situation:

While building Apex, generation was working: the app could take a prompt, generate CDK, run cdk synth in the sandbox, and show diff, cost, and security results. But when I clicked Approve & Deploy, the UI showed deployment failure/rollback. At first it looked like a frontend button or fetch issue, because the user-facing symptom was just "deploy failed" or sometimes a misleading "Failed to fetch" toast.

Task:

My job was to figure out whether the problem was in the frontend, the approval API, the deploy Lambda, CloudFormation, or IAM, and fix it without just giving broad AWS permissions to AI-generated infrastructure.

Action:

I traced the request end to end. The button was correctly calling the approval endpoint, the approval Lambda was updating DynamoDB to deploying, and it was async-invoking the deploy Lambda. Then I checked the CloudFormation stack events and found the actual failure: the generated template included a customer-managed KMS key for the S3 bucket, but CloudFormation was deploying under a scoped DeploymentRole that originally supported only S3 actions. So sandbox validation passed because cdk synth only checks whether the CDK compiles into CloudFormation, not whether the deployment role has runtime permissions like kms:CreateKey.

I fixed this in two ways. First, I added the minimum KMS permissions needed for the supported S3-with-KMS use case. Second, I added a deployability guardrail before creating the CloudFormation change set: the deploy Lambda now inspects the template resource types and blocks unsupported resources with a clear deploy_failed message instead of letting CloudFormation fail halfway through. I also made sure failure paths update DynamoDB terminally so records do not stay stuck in deploying, and added tests for successful deploys, rollback handling, no-op change sets, retry, and unsupported resource blocking.

Result:

After the fix, the S3 + KMS example could go from awaiting_approval to deploying to deployed, with live CloudFormation events streamed back to the UI. More importantly, the system became safer and more explainable: supported resources deploy, unsupported resources fail fast with a friendly reason, and the app no longer hides an IAM mismatch behind a generic deploy-button failure.

Polished Version To Say Aloud

> *One technical challenge I faced was when the "Approve & Deploy" button appeared broken. The generation pipeline worked, but after approval the deployment rolled back. I traced the flow from the frontend to the approval Lambda, DynamoDB, the async deploy Lambda, and finally CloudFormation events. The root cause was not the button at all: the generated CDK created an S3 bucket encrypted with a customer-managed KMS key, but CloudFormation was running under a scoped deployment role that only had S3 permissions. The sandbox passed because cdk synth validates template generation, not deploy-time IAM. I fixed it by adding the minimum KMS permissions for that supported use case and by adding a pre-deploy resource allowlist guardrail, so unsupported templates fail fast before CloudFormation. I also added tests around rollback, retry, no-change deployments, and unsupported resources. The result was a more reliable deploy flow and a clearer safety boundary between AI generation, validation, approval, and real AWS provisioning.*

Likely Follow-Ups

- Q: Why didn't sandbox validation catch this? A: Because sandbox validation runs cdk synth. It proves the generated TypeScript can synthesize into CloudFormation, but it does not prove the deployment role can create every resource.
- Q: Why not just give the deployment role admin permissions? A: Because the infrastructure is AI-generated. I wanted a controlled blast radius, so CloudFormation runs under an allowlisted role. I expanded permissions deliberately only for supported resources.
- Q: What was the key lesson? A: There are multiple layers of validation: syntax/synthesis validation, security scanning, deployability checks, and actual CloudFormation execution. Passing one layer does not guarantee the next.